



优化程序性能 - 编译相关的优化技术

2400012960 祁优扬

目录

- 前言
- 通用的编译优化
 - 代码移动 (Code Motion / Precomputation)
 - 强度折减 (Strength Reduction)
 - 共用公共子表达式 (Sharing of Common Subexpressions)
 - 一个例子：冒泡排序
- 编译优化的限制 (Optimization Blockers)
 - 函数调用 (Procedure Calls)
 - 内存别名 (Memory Aliasing)
- 小结

前言

当我们衡量一个程序的运行效率时，我们要考虑什么？

- 渐进时间复杂度
- 常数因子

时间复杂度当然重要，但常数也必须纳入考虑。后面涉及的优化大部分都是常数优化。

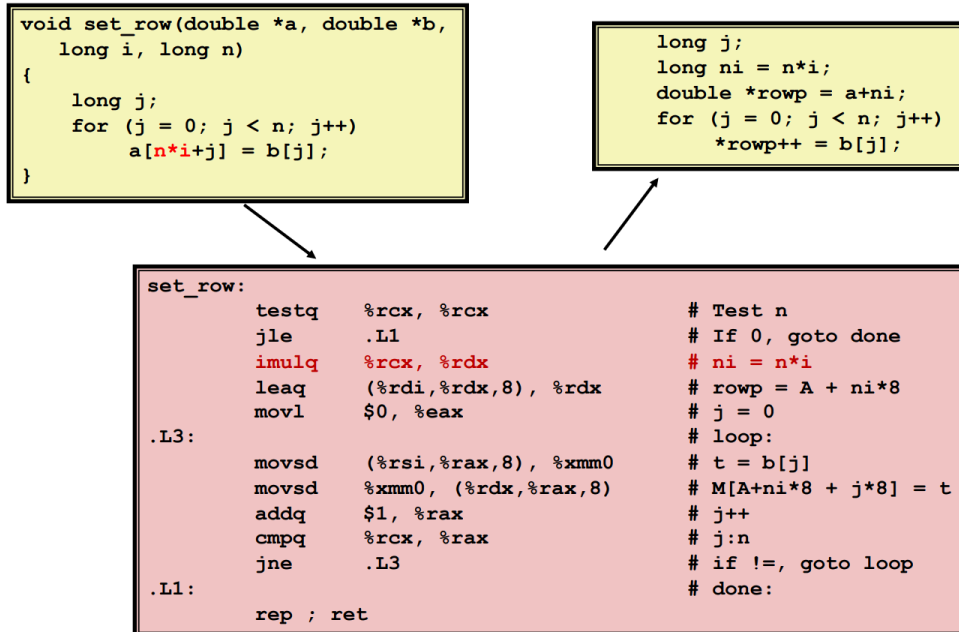
通用的编译优化

代码移动 (Code Motion / Precomputation)

当某个计算执行多次并产生相同结果时，只实际执行一次，减少重复计算。

常见的情况是将代码移出循环，比如课件中的例子，`n*i` 是在循环体之前计算的。

Compiler-Generated Code Motion (-O1)



另外一个例子如下。

C 代码：

```
1 void test_code_motion(int *a, long n, long m) {
2     for(long i = 0; i < n * m; i++) {
3         a[i] = 114514;
4     }
5 }
```

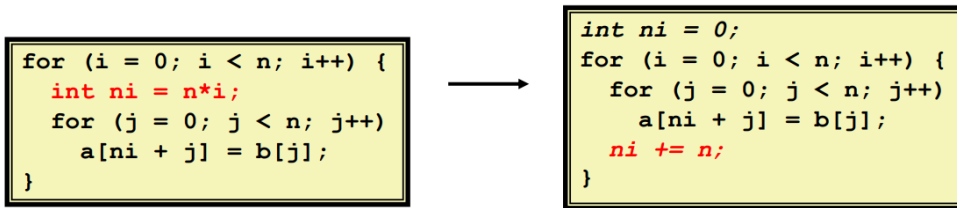
汇编代码（-O1）：

```
7 # a in %rdi, n in %rsi, m in %rdx
8 0000000000000000 <test_code_motion>:
9 0: f3 0f 1e fa          endbr64
10 4: 48 0f af f2          imul    %rdx,%rsi          # %rsi <- n*m
11 8: 48 85 f6             test    %rsi,%rsi
12 b: 7e 16               jle     23 <test_code_motion+0x23> # guarded do
13 d: 48 89 f8             mov     %rdi,%rax          # %rax <- a
14 10: 48 8d 14 b7          lea     (%rdi,%rsi,4),%rdx  # %rdx <- a+n*m*4
15 14: c7 00 52 bf 01 00    movl    $0x1bf52,(%rax)    # (%rax) <- 114514
16 1a: 48 83 c0 04          add     $0x4,%rax          # %rax = %rax + 4
17 1e: 48 39 d0             cmp     %rdx,%rax          # %rax - %rdx
18 21: 75 f1               jne     14 <test_code_motion+0x14> # loop
19 23: c3                  ret
20
```

可以看到，编译出的汇编代码中，循环比较中的 `n*m` 是在循环体之前计算的。

强度折减 (Strength Reduction)

把代价比较大的操作（如乘法）替换成更简单的操作。



我们可以测试编译器是否会自动进行这项优化。

C 代码：

```
1 void test_strength_reduction(long *a, long *b, long n) {
2     for(long i = 0; i < n; i++) {
3         long ni = n * i;
4         for(long j = 0; j < n; j++)
5             a[ni + j] = b[j];
6     }
7 }
```

汇编代码（-O1）：

```
7 # a in %rdi, b in %rsi, n in %rdx
8 0000000000000000 <test_strength_reduction>:
9 0: f3 0f 1e fa          endbr64
10 4: 48 85 d2             test    %rdx,%rdx
11 7: 7e 38               jle     41 <test_strength_reduction+0x41>
12 9: 4c 8d 0c d5 00 00 00 lea     0x0(,%rdx,8),%r9          # %r9 <- n*8
13 10: 00
14 11: 41 b8 00 00 00 00     mov     $0x0,%r8d
15 17: eb 03               jmp     1c <test_strength_reduction+0x1c>
16 19: 49 89 c0             mov     %rax,%r8
17 1c: b8 00 00 00 00     mov     $0x0,%eax
18 21: 48 8b 0c c6         mov     (%rsi,%rax,8),%rcx
19 25: 48 89 0c c7         mov     %rcx,(%rdi,%rax,8)
20 29: 48 89 c1             mov     %rax,%rcx
21 2c: 48 83 c0 01         add     $0x1,%rax
22 30: 48 39 c2             cmp     %rax,%rdx
23 33: 75 ec               jne     21 <test_strength_reduction+0x21>
24 35: 49 8d 40 01         lea     0x1(%r8),%rax
25 39: 4c 01 cf         add     %r9,%rdi          # %rdi += %r9
26 3c: 49 39 c8             cmp     %rcx,%r8
27 3f: 75 d8               jne     19 <test_strength_reduction+0x19>
28 41: c3                 ret
```

可以看到，编译出的汇编代码中，并没有出现乘法计算，`long ni = n * i` 的乘法计算被加法代替了。

共用公共子表达式 (Sharing of Common Subexpressions)

复用相似表达式的公共部分。在开启 `-O1` 及以上级别的优化时，编译器会自动完成这一优化。在课件中的例子中，通过提前计算子表达式 `i*n + j`，将乘法次数从 3 降到了 1。

```
/* Sum neighbors of i,j */
up = val[(i-1)*n + j];
down = val[(i+1)*n + j];
left = val[i*n + j-1];
right = val[i*n + j+1];
sum = up + down + left + right;
```

3 multiplications: `i*n`, `(i-1)*n`, `(i+1)*n`

```
long inj = i*n + j;
up = val[inj - n];
down = val[inj + n];
left = val[inj - 1];
right = val[inj + 1];
sum = up + down + left + right;
```

1 multiplication: `i*n`

```
leaq 1(%rsi), %rax # i+1
leaq -1(%rsi), %r8 # i-1
imulq %rcx, %rsi # i*n
imulq %rcx, %rax # (i+1)*n
imulq %rcx, %r8 # (i-1)*n
addq %rdx, %rsi # i*n+j
addq %rdx, %rax # (i+1)*n+j
addq %rdx, %r8 # (i-1)*n+j
...
```

```
imulq %rcx, %rsi # i*n
addq %rdx, %rsi # i*n+j
movq %rsi, %rax # i*n+j
subq %rcx, %rax # i*n+j-n
leaq (%rsi,%rcx), %rcx # i*n+j+n
...
```

另外一个例子如下。

C 代码：

```
1 long test_subexpression(long *a, long n, long i) {
2     long left = a[n*i-1], right = a[n*i+1];
3     return left + right;
4 }
```

汇编代码 (`-O1`)：

```
7 0000000000000000 <test_subexpression>:
8 0: f3 0f 1e fa          endbr64
9 4: 48 0f af f2          imul %rdx,%rsi
10 8: 48 8b 44 f7 f8       mov -0x8(%rdi,%rsi,8),%rax
11 d: 48 03 44 f7 08       add 0x8(%rdi,%rsi,8),%rax
12 12: c3                  ret
```

可以看到，`n*i` 作为公共子表达式被复用了。

一个例子：冒泡排序

以冒泡排序为例，我们可以分析它的优化过程。

这是冒泡排序的 C 代码以及它所对应的细化后的伪代码。

Translated (Pseudo) Code

```
i := n-1
L5: if i<1 goto L1
j := 1
L4: if j>i goto L2
t1 := j-1
t2 := 4*t1
t3 := A[t2] // A[j]
t4 := j+1
t5 := t4-1
t6 := 4*t5
t7 := A[t6] // A[j+1]
if t3<=t7 goto L3
```

```
for (i = n-1; i >= 1; i--) {
  for (j = 1; j <= i; j++)
    if (A[j] > A[j+1]) {
      temp = A[j];
      A[j] = A[j+1];
      A[j+1] = temp;
    }
}
```

```
t8 := j-1
t9 := 4*t8
temp := A[t9] // temp:=A[j]
t10 := j+1
t11:= t10-1
t12 := 4*t11
t13 := A[t12] // A[j+1]
t14 := j-1
t15 := 4*t14
A[t15] := t13 // A[j]:=A[j+1]
t16 := j+1
t17 := t16-1
t18 := 4*t17
A[t18]:=temp // A[j+1]:=temp
L3: j := j+1
goto L4
L2: i := i-1
goto L5
L1:
```

Instructions
29 in outer loop
25 in inner loop

可以看到，在地址计算中，存在着很多冗余 (Redundancy)。

Redundancy in Address Calculation

```
i := n-1
L5: if i<1 goto L1
j := 1
L4: if j>i goto L2
t1 := j-1
t2 := 4*t1
t3 := A[t2] // A[j]
t4 := j+1
t5 := t4-1
t6 := 4*t5
t7 := A[t6] // A[j+1]
if t3<=t7 goto L3
```

```
t8 :=j-1
t9 := 4*t8
temp := A[t9] // temp:=A[j]
t10 := j+1
t11:= t10-1
t12 := 4*t11
t13 := A[t12] // A[j+1]
t14 := j-1
t15 := 4*t14
A[t15] := t13 // A[j]:=A[j+1]
t16 := j+1
t17 := t16-1
t18 := 4*t17
A[t18]:=temp // A[j+1]:=temp
L3: j := j+1
goto L4
L2: i := i-1
goto L5
L1:
```

t5 的计算完全是多余的：执行完 t4 := j+1 和 t5 := t4-1 后，t5 就等于 j；t11 同理。t15 的计算过程与 t9 完全相同，也是多余的；t18 同理。

初步去除冗余后，我们可以得到下面的伪代码。

Redundancy Removed

```
i := n-1
L5: if i<1 goto L1
    j := 1
L4: if j>i goto L2
    t1 := j-1
    t2 := 4*t1
    t3 := A[t2]    // A[j]
    t6 := 4*j
    t7 := A[t6]    // A[j+1]
    if t3<=t7 goto L3
    t8 := j-1
    t9 := 4*t8
    temp := A[t9]  // temp:=A[j]
    t12 := 4*j
    t13 := A[t12]  // A[j+1]
    A[t9] := t13   // A[j]:=A[j+1]
    A[t12] := temp // A[j+1]:=temp
L3: j := j+1
    goto L4
L2: i := i-1
    goto L5
L1:
```

Instructions
20 in outer loop
16 in inner loop

我们还可以观察到更多的冗余。

More Redundancy

```
i := n-1
L5: if i<1 goto L1
    j := 1
L4: if j>i goto L2
    t1 := j-1
    t2 := 4*t1
    t3 := A[t2]    // A[j]
    t6 := 4*j
    t7 := A[t6]    // A[j+1]
    if t3<=t7 goto L3
    t8 := j-1
    t9 := 4*t8
    temp := A[t9]  // temp:=A[j]
    t12 := 4*j
    t13 := A[t12]  // A[j+1]
    A[t9] := t13   // A[j]:=A[j+1]
    A[t12] := temp // A[j+1]:=temp
L3: j := j+1
    goto L4
L2: i := i-1
    goto L5
L1:
```

`A[j]` 和 `A[j+1]` 的值在之前进行大小比较时已经取出，不必再取一次。

再次去除冗余后，我们可以得到下面的伪代码。

Redundancy Removed

```
    i := n-1
L5: if i<1 goto L1
    j := 1
L4: if j>i goto L2
    t1 := j-1
    t2 := 4*t1
    t3 := A[t2]    // old_A[j]
    t6 := 4*j
    t7 := A[t6]    // A[j+1]
    if t3<=t7 goto L3
                                L3: j := j+1
                                goto L4
                                L2: i := i-1
                                goto L5
                                L1:
```

Instructions
15 in outer loop
11 in inner loop

循环过程中仍然存在冗余。

Redundancy in Loops

```
    i := n-1
L5: if i<1 goto L1
    j := 1
L4: if j>i goto L2
    t1 := j-1
    t2 := 4*t1
    t3 := A[t2]    // A[j]
    t6 := 4*j
    t7 := A[t6]    // A[j+1]
    if t3<=t7 goto L3
    A[t2] := t7
    A[t6] := t3
L3: j := j+1
    goto L4
L2: i := i-1
    goto L5
L1:
```

直接把循环变量的步长改为 4，可以去除大部分的乘法操作。

修改后的伪代码如下。

Redundancy Eliminated

```
    i := n-1
L5: if i<1 goto L1
    j := 1
L4: if j>i goto L2
    t1 := j-1
    t2 := 4*t1
    t3 := A[t2]    // A[j]
    t6 := 4*j
    t7 := A[t6]    // A[j+1]
    if t3<=t7 goto L3
    A[t2] := t7
    A[t6] := t3
L3: j := j+1
    goto L4
L2: i := i-1
    goto L5
L1:
```

```
    i := n-1
L5: if i<1 goto L1
    t2 := 0
    t6 := 4
    t19 := 4*i
L4: if t6>t19 goto L2
    t3 := A[t2]
    t7 := A[t6]
    if t3<=t7 goto L3
    A[t2] := t7
    A[t6] := t3
L3: t2 := t2+4
    t6 := t6+4
    goto L4
L2: i := i-1
    goto L5
L1:
```

最终的伪代码如下。可以看到，内外层每轮循环的指令数都大幅度下降，运行效率从而大幅度提高。我们可以用伪代码的优化过程类比编译器在编译中的优化过程。

Final Pseudo Code

```
    i := n-1
L5: if i<1 goto L1
    t2 := 0
    t6 := 4
    t19 := i << 2
L4: if t6>t19 goto L2
    t3 := A[t2]
    t7 := A[t6]
    if t3<=t7 goto L3
    A[t2] := t7
    A[t6] := t3
L3: t2 := t2+4
    t6 := t6+4
    goto L4
L2: i := i-1
    goto L5
L1:
```

Instruction Count Before Optimizations

29 in outer loop
25 in inner loop

Instruction Count After Optimizations

15 in outer loop
9 in inner loop

- These were **Machine-Independent Optimizations**.
- Will be followed by **Machine-Dependent Optimizations**, including allocating temporaries to registers, converting to assembly code

这些优化都可以在伪代码层面体现出来，是不依赖机器（Machine Independent）的。

编译优化的限制 (Optimization Blockers)

编译优化**不能**改变程序行为。

- 当然，前提是程序符合语言标准。编译优化可能改变未定义行为 (Undefined Behavior, UB) 的运行结果，如有符号整数溢出、数组越界访问、访问未初始化变量、`(x++ * ++x)` 等。

当编译器无法保证优化不会改变程序行为时，编译器会采取**保守策略**，往往就是不进行相应的优化。

以下是两种常见的限制编译优化的因素，函数调用 (Procedure Calls) 和内存别名 (Memory Aliasing) 。

函数调用 (Procedure Calls)

以下代码将字符串 `s` 中的所有大写字母转成小写字母。

```
// 优化前
void lower(char *s) {
    size_t i;
    for(i = 0; i < strlen(s); i++)
        if(s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}
```

在实际的测试中，它的运行效率非常缓慢——在每轮循环中，都会调用 `strlen` 函数，这个函数的工作方式是向后遍历整个字符串，直到找到 `\0` 字符。

这导致它的时间复杂度变为 $O(n^2)$ ，而原本在 $O(n)$ 的时间里即可完成这一任务，只要简单地把 `strlen(s)` 提前计算好。

```
// 优化后
void lower(char *s) {
    size_t i;
    size_t len = strlen(s);
    for(i = 0; i < len; i++)
        if(s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}
```

这里我们其实就是手动完成了之前所说的 Code Motion。因为调用了函数 `strlen`，所以编译器不会自动完成这项优化。

函数可能会有副作用（改变某些全局状态），也可能在参数相同的不同调用中返回不同的结果（依赖于某些全局状态等）。

因此，编译器把函数调用视为黑箱。涉及函数调用时，编译器能进行的优化非常有限。

但我们可以用**内联函数替换**（Inline Substitution）的方式进行优化，将函数调用替换为函数体。

比如，在下面的代码中，`func1` 调用了 `f`。

```
long cnt = 0;
long f() {
    return cnt++;
}
long func1() {
    return f() + f() + f() + f();
}
```

内联函数替换后得到的结果如下。

```
long func1in() {
    long t = cnt++;
    t += cnt++;
    t += cnt++;
    t += cnt++;
    return t;
}
```

对 `func1in`，编译器就可以进行优化了。

```
long func1opt() {
    long t = 4 * cnt + 6;
    cnt += 4;
    return t;
}
```

GCC 的最近版本会尝试进行这种形式的优化（使用命令行选项 `-finline` 或开启 `-O1` 及以上级别的优化时）。

- 不过，GCC 只尝试在单个文件中定义的函数的内联，不进行跨文件的内联。

内存别名 (Memory Aliasing)

下面的函数对 `n×n` 矩阵 `a` 的每一行求和并依次存储到 `b` 中。（是这样吗？）

```
void sum_rows1(double *a, double *b, long n) {
    long i, j;
    for(i = 0; i < n; i++) {
        b[i] = 0;
        for(j = 0; j < n; j++)
            b[i] += a[i*n + j];
    }
}
```

编译出的汇编代码如下。

```
# sum_rows1 inner loop
.L4:
    movsd (%rsi,%rax,8), %xmm0 # FP load
    addsd (%rdi), %xmm0        # FP add
    movsd %xmm0, (%rsi,%rax,8) # FP store
    addq $8, %rdi
    cmpq %rcx, %rdi
    jne .L4
```

为什么编译器不使用一个浮点寄存器暂存 `b[i]`，来减少内存加载的次数呢？

实际上，`a` 和 `b` 的某些部分可能是重合的（这被称为**内存别名**，即**两个不同的内存引用指定了同一个内存位置**），如上所述的优化可能改变运行结果，所以编译器不会进行这一优化。

我们可以手动优化。

```
void sum_rows2(double *a, double *b, long n) {
    long i, j;
    for(i = 0; i < n; i++) {
        double val = 0;
        for(j = 0; j < n; j++)
            val += a[i*n + j];
        b[i] = val;
    }
}
```

编译出的汇编代码如下。

```
# sum_rows2 inner loop
.L10:
    addsd (%rdi), %xmm0 # FP load + add
    addq $8, %rdi
    cmpq %rax, %rdi
    jne .L10
```

我们在编写 C 语言代码的时候知道（或者说，会在调用 `sum_rows2` 时保证），数组 `a` 与 `b` 在内存中没有重叠部分，所以进行这一优化不会影响程序行为。编译器不知道这一点。

所以，在写代码时要记得引入局部变量，这其实是在告诉编译器不用考虑 Memory Aliasing。

小结

- 通用的编译优化
 - 代码移动（Code Motion / Precomputation）
 - 强度折减（Strength Reduction）
 - 共用公共子表达式（Sharing of Common Subexpressions）
 - 一个例子：冒泡排序
- 编译优化的限制（Optimization Blockers）
 - 函数调用（Procedure Calls）

- 内存别名 (Memory Aliasing)
- When in doubt, the compiler must be **conservative**.